TECHNICAL HANDOFF

CODER BRIEF.

Architecture, schema, realtime model, and production gaps. Everything you need to understand the prototype and spec the production build.

01 — STACK

What it's built on.

Zero build step. Vanilla JS module in a single HTML file, deployed as a static asset on Vercel.

- **Frontend:** Single `index.html`. No framework, no bundler, no npm. Supabase client pulled via `esm.sh` CDN import.
- **Database + Realtime:** Supabase (Postgres). Three tables. Realtime via `postgres_changes` subscription over a persistent WebSocket.
- **Hosting:** Vercel. Auto-deploys on push to `main`. Static file serve — no serverless functions in use.
- **Repo:** `github.com/JaroGee/infection-drop-demo`
- **Live URL:** `infection-drop-demo.vercel.app`

```
// Supabase client — hardcoded in index.html
const SUPABASE_URL = 'https://aeaylyimninfdkhkusoc.supabase.co';
const SUPABASE_KEY = 'sb_publishable_t-KRtim6z-JJLopMFrBs1A_5soEwoey';
const supabase = createClient(SUPABASE_URL, SUPABASE_KEY);
```

```
// Module import — no bundler needed
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2.45.4';
```

> NO RLS
>
> Row-level security is off on all three tables. Any client with the publishable key can read and write anything. Fine for a closed demo; needs to be locked down before any public-facing build.

---

02 — SCHEMA

Three tables.

cells

COLUMN	TYPE	NOTES
id	uuid PK	Auto-generated. Used as the FK reference for players and decisions.
code	text	The user-visible cell code (e.g. `DEMO`). Unique. Lookup key on join.
name	text	Human label, set to `{code} Cell` on creation. Not currently displayed.
current_day	int4	Starts at 1. Incremented by any player clicking Advance Day. Realtime UPDATE on this column triggers day transitions across all connected clients. When > 7, all clients route to the Reckoning screen.
created_at	timestamptz	Auto. Unused in app logic.

players

COLUMN	TYPE	NOTES
id	uuid PK	Stored in `sessionStorage` as `idDemo.playerId` to resume sessions after refresh.
cell_id	uuid FK	References `cells.id`. Subscription filter is `cell_id=eq.{cellId}`.
display_name	text	Same as initial in demo. In production, this would be the verified name/handle.

initial	text	Single char. The avatar shown in member pills. Uppercased on input.
is_active	bool	Defaults true. Players query filters on `is_active=true`. Soft-delete path, not currently used.
joined_at	timestamptz	Auto. Used to order member pills (oldest first).

decisions

COLUMN	TYPE	NOTES
id	uuid PK	Auto.
player_id	uuid FK	References `players.id`.
cell_id	uuid FK	Denormalised for simpler queries and subscription filtering.
day	int4	1–7. Combined with `player_id` forms a logical unique key (not enforced as constraint in demo).
choice	text	`'HOLD'` or `'RELEASE'`. Enum not enforced at DB level in demo.
created_at	timestamptz	Auto.

---

03 — REALTIME

How sync works.

One Supabase channel per cell. Three `postgres_changes` listeners. Any DB mutation triggers a full reload of client-side state followed by a full re-render.

```
state.channel = supabase
  .channel('cell:' + state.cellId)
  .on('postgres_changes',
    { event: '*', schema: 'public', table: 'players',
      filter: 'cell_id=eq.' + state.cellId },
    async () => { await loadState(); renderAll(); }
  )
  .on('postgres_changes',
    { event: '*', schema: 'public', table: 'decisions',
      filter: 'cell_id=eq.' + state.cellId },
    async () => { await loadState(); renderAll(); }
```

```
  )
  .on('postgres_changes',
    { event: 'UPDATE', schema: 'public', table: 'cells',
      filter: 'id=eq.' + state.cellId },
    async (payload) => {
      state.currentDay = payload.new.current_day;
      await loadState(); renderAll();
      if (state.currentDay > 7) { show('screen-reckoning'); renderReckoning(); }
    }
  )
  .subscribe();
```

- **Players listener:** fires on INSERT/UPDATE/DELETE. Used when someone new joins the cell mid-demo.
- **Decisions listener:** fires on INSERT. The main sync path — when one client taps HOLD/RELEASE, all others update within ~100–300ms on a warm connection.
- **Cells listener:** fires on UPDATE only. Day advancement and Reckoning routing live here. Because `current_day` comes directly from `payload.new`, this avoids a round-trip for the field that matters most.
- **No optimistic updates.** UI only changes after the DB write succeeds and the realtime event comes back. Keeps state consistent at the cost of a small perceived latency on the deciding player's own phone.

> REALTIME ON SUPABASE FREE TIER
>
> Free plan supports 200 concurrent realtime connections and 2M messages/month. More than enough for a demo. For a 200-person live event, verify concurrent connection limits against the org's current plan before the build.

---

04 — CLIENT STATE

The state object and screen flow.

```
// Single mutable object. No framework state management.
let state = {
  playerId:   null,   // uuid — set on join, persisted to sessionStorage
  cellId:     null,   // uuid — FK used in all queries and subscriptions
  cellCode:   null,   // string — shown in presence indicator
  initial:    null,   // string — the player's single-char avatar
  currentDay: 1,      // int 1–8 (>7 = Reckoning)
```

```
  players:    [],     // [{id, initial, display_name, is_active}]
  decisions:  [],     // [{player_id, cell_id, day, choice}]
  channel:    null,   // RealtimeChannel — stored to unsubscribe on reset
};
```

Three screens, toggled by the `show()` helper which adds/removes `.hidden`:

- **`screen-onboard`** — initial + cell code input. On submit: find-or-create cell row, insert player row, write to `sessionStorage`, `loadState()`, `subscribeRealtime()`.
- **`screen-game`** — the daily loop. Renders story, infection meter, member pills, HOLD/RELEASE buttons, sub-actions, progress ticks. Driven entirely by `renderAll()` reading from `state`.
- **`screen-reckoning`** — final screen. Shown when `currentDay > 7`, either from realtime UPDATE callback or on resume from `sessionStorage`. Contains the REPLAY DEMO button.

Session resume

On page load, the script checks `sessionStorage.getItem('idDemo')`. If present, it re-hydrates `state`, calls `loadState()` to pull fresh DB data, and re-subscribes. This means a phone can refresh mid-game without losing its place. The session survives tab closes but not private browsing or manual clear.

---

05 — RESET

How REPLAY DEMO works.

The reset is a server-side wipe, not just a client-side state clear. Three Supabase operations fire in parallel before the UI resets to onboarding.

```
document.getElementById('btn-restart').addEventListener('click', async () => {
  const cellId = state.cellId;              // capture before wipe
  sessionStorage.removeItem('idDemo');
  if (state.channel) state.channel.unsubscribe();
  state = { playerId: null, cellId: null, cellCode: null, initial: null,
            currentDay: 1, players: [], decisions: [], channel: null };

  if (cellId) {
    await Promise.all([
      supabase.from('decisions').delete().eq('cell_id', cellId),
      supabase.from('players').delete().eq('cell_id', cellId),
```

```
      supabase.from('cells').update({ current_day: 1 }).eq('id', cellId),
    ]);
  }
  // then reset UI to onboard screen
});
```

- Captures `cellId` before wiping `state` — the Promise.all needs it.
- Unsubscribes the realtime channel before state wipe, avoiding orphaned listeners.
- Parallel deletes: decisions and players are independent. Cell update runs alongside them. `await Promise.all` blocks UI reset until all three complete.
- After reset, the same cell *code* (e.g. `DEMO`) can be re-used — the cell row still exists with `current_day = 1`. New players will be inserted fresh on the next join.

> RACE CONDITION NOTE
>
> If two clients are connected and one hits REPLAY DEMO while the other is mid-decision, the deletes will wipe the second client's data. The second client will receive realtime events for the deletes and re-render an empty state — it won't crash, but the experience is abrupt. Not worth fixing for a controlled demo; worth addressing in production with a lobby/host model.

---

06 — INFECTION METER

How the percentage is calculated.

Purely client-side. Derived from `state.decisions` on every render. Not stored in the DB.

```
function calculateInfection() {
  const totalReleases = state.decisions.filter(d => d.choice === 'RELEASE').length;
  const totalHolds    = state.decisions.filter(d => d.choice === 'HOLD').length;
  const cellSize      = Math.max(state.players.length, 1);
  const dayMultiplier = state.currentDay * 2;     // baseline rises each day
  const base          = 30 + dayMultiplier;        // Day 1 = 32%, Day 7 = 44%
  const releaseEffect = (totalReleases / cellSize) * 12;
  const holdEffect    = (totalHolds / cellSize) * 6;
  let pct = base + releaseEffect - holdEffect;
  return Math.max(8, Math.min(96, Math.round(pct))); // clamp 8–96
}
```

- Baseline grows by 2pp per day regardless of decisions — conveys that pressure accumulates over time.
- Each RELEASE adds `12 / cellSize` — effect is diluted in larger cells.
- Each HOLD subtracts `6 / cellSize` — holds soften but don't cancel releases at equal count.
- Clamped to 8–96 so it never reads as a clean zero or absolute maximum, which would feel wrong.
- State labels: `< 40%` = HOLDING, `< 65%` = PRESSURED, `< 85%` = BREACHING, else CRITICAL.

---

07 — DEMO SHORTCUTS

What's hardcoded or missing.

- **Stories are hardcoded:** `const stories = { 1: "...", ..., 7: "..." }` in the script. No CMS, no DB fetch. Change the strings directly to update content.
- **Any player can advance the day:** `btn-advance` is visible to all. In production this should be host-only or triggered by a facilitator dashboard.
- **Any player can reset:** REPLAY DEMO is visible to everyone on the Reckoning screen. Same host-only caveat.
- **Cell cap not enforced:** UI shows `1/6` max but there's no DB constraint. Six or more players can join the same cell code.
- **No duplicate decision guard at DB level:** A player can technically INSERT two decisions for the same day if they race. The client checks `getMyDecisionToday()` and returns early, but there's no unique constraint on `(player_id, day)`.
- **RESCUE and INVITE are placeholder UI:** RESCUE shows a toast. INVITE uses `navigator.share` or clipboard. Neither writes to the DB.
- **Cell rank is hardcoded:** `state.players.length >= 4 ? '12' : '—'`. Not a real leaderboard.
- **URL param autofill:** `?cell=CODE` pre-fills the cell code input. Useful for invite links. Already in production — `btn-invite` generates this URL.
- **No auth:** Players are anonymous. `playerId` is a raw UUID stored in `sessionStorage`. No Supabase Auth in use.

---

08 — PRODUCTION BUILD

What needs to be built from scratch.

The demo covers the player-side daily loop only. Production is a different scope. Technical breakdown of the open items:

- **Auth + consent:** Phone number OTP via Supabase Auth or Twilio Verify. Consent record tied to `auth.users.id`. Age-gate logic (date-of-birth capture, under-16 parental consent flow).
- **Host / facilitator role:** Separate auth claim for event staff. Controls day advancement, cell creation, reset, and broadcast trigger. Prevents participants from manipulating game state.
- **RLS on all tables:** Players can only read/write their own cell's data. Decisions insert-only once per `(player_id, day)` — add unique constraint. Cells update restricted to host role.
- **Content pipeline:** Stories, scenarios, and copy pulled from a DB table or CMS. Allows last-minute editorial changes without a redeploy.
- **The Confessional:** Voice recording capture (MediaRecorder API or native on PWA), upload to Supabase Storage, pitch-shift / modulation pass (possible client-side via Web Audio API), moderation queue before broadcast inclusion.
- **Reckoning live vote:** Simultaneous poll with instant aggregation. Supabase Realtime already handles the sync pattern — the main lift is the broadcast UI and the aggregation query under load.
- ***SCAPE plaza integration:** Sub-200ms input-to-display target means the realtime subscription must stay on a warm connection. Consider a dedicated Supabase channel for the broadcast display vs. individual player channels.
- **AI moderation:** Pipe user-submitted content through a moderation API before surfacing publicly. Human escalation queue for flagged items.
- **Evaluation / reporting:** Pre/post survey flows, decision export, nightly digest emails, final NCADA-format report. Likely a separate internal dashboard, not part of the participant app.
- **PWA shell:** Add manifest + service worker for homescreen install and offline resilience. Not strictly required but expected UX for a week-long daily engagement.
- **Push notifications:** Daily prompt delivery. Web Push API or a native wrapper. Requires Notification permission — factor into the onboarding consent flow.

> STACK RECOMMENDATION FOR PRODUCTION
>
> The prototype's vanilla-JS-on-Vercel approach works fine for a 6-player demo. For a 200-player live event with a facilitator dashboard, push notifications, voice recording, and a broadcast display, the build warrants a proper framework (Next.js fits the Vercel stack) and Supabase row-level security enabled from the start. Don't grow the prototype — build the production version clean.